

```

function [x_opt, fval, history] = simplex_search(f, x0, varargin)
p = inputParser;
addRequired(p, 'f');
addRequired(p, 'x0');
addParameter(p, 'MaxIter', 1000);
addParameter(p, 'TolFun', 1e-6);
addParameter(p, 'TolX', 1e-6);
addParameter(p, 'Display', 'iter');
parse(p, f, x0, varargin{:});

max_iter = p.Results.MaxIter;
tol_fun = p.Results.TolFun;
tol_x = p.Results.TolX;
display = p.Results.Display;

n = length(x0);
alpha = 1.0;
gamma = 2.0;
rho = 0.5;
sigma = 0.5;

simplex = initialize_simplex(x0);
f_simplex = zeros(n+1, 1);
for i = 1:n+1
    f_simplex(i) = f(simplex(i,:));
end

history.x = zeros(max_iter, n);
history.fval = zeros(max_iter, 1);
history.simplex = cell(max_iter, 1);

iter = 0;
converged = false;

if strcmp(display, 'iter')
    fprintf('Iter\tFunc-count\tMin f(x)\tCurrent\n');
end

while iter < max_iter && ~converged
    iter = iter + 1;

    [f_simplex, order] = sort(f_simplex);
    simplex = simplex(order, :);

    history.x(iter,:) = simplex(1,:);
    history.fval(iter) = f_simplex(1);
    history.simplex{iter} = simplex;

    if strcmp(display, 'iter')

```

```

        fprintf('%d\t%d\t%.6e\t%.6e\n', iter, n+1, f_simplex(1),
norm(simplex(2:end,:) - simplex(1,:), 'fro'));
    end

    if std(f_simplex) < tol_fun && norm(simplex(2:end,:) - simplex(1,:), 'fro') <
tol_x
        converged = true;
        break;
    end

    centroid = mean(simplex(1:end-1,:), 1);

    xr = centroid + alpha * (centroid - simplex(end,:));
    fxr = f(xr);

    if f_simplex(1) <= fxr && fxr < f_simplex(end-1)
        simplex(end,:) = xr;
        f_simplex(end) = fxr;

    elseif fxr < f_simplex(1)
        xe = centroid + gamma * (xr - centroid);
        fxe = f(xe);

        if fxe < fxr
            simplex(end,:) = xe;
            f_simplex(end) = fxe;
        else
            simplex(end,:) = xr;
            f_simplex(end) = fxr;
        end

    else
        if fxr < f_simplex(end)
            xc = centroid + rho * (xr - centroid);
        else
            xc = centroid + rho * (simplex(end,:) - centroid);
        end
        fxc = f(xc);

        if fxc < f_simplex(end)
            simplex(end,:) = xc;
            f_simplex(end) = fxc;
        else
            for i = 2:n+1
                simplex(i,:) = simplex(1,:) + sigma * (simplex(i,:) - simplex(1,:));
                f_simplex(i) = f(simplex(i,:));
            end
        end
    end
end
end

```

```

history.x = history.x(1:iter,:);
history.fval = history.fval(1:iter);
history.simplex = history.simplex(1:iter);

x_opt = simplex(1,:);
fval = f_simplex(1);

if strcmp(display, 'iter') || strcmp(display, 'final')
    if converged
        fprintf('Optimization converged after %d iterations\n', iter);
    else
        fprintf('Maximum iterations reached (%d)\n', max_iter);
    end
    fprintf('Final objective value: %.6e\n', fval);
    fprintf('Optimal point: ');
    fprintf('%.6f ', x_opt);
    fprintf('\n');
end

end

function simplex = initialize_simplex(x0)
n = length(x0);
simplex = zeros(n+1, n);
simplex(1,:) = x0;
for i = 1:n
    direction = zeros(1, n);
    direction(i) = 0.05;
    if x0(i) == 0
        direction(i) = 0.00025;
    else
        direction(i) = 0.05 * x0(i);
    end
    simplex(i+1,:) = x0 + direction;
end
end

```

## question1

```

clc; clear; close all;

f = @(x) x(1)^2 - 2*x(1)*x(2) - 19*x(1) + 7*x(2)^2;
x0 = [0.5, 1.5];

[x_opt, fval] = fminsearch(f, x0, optimset('MaxIter', 3));

```

Exiting: Maximum number of iterations has been exceeded

- increase MaxIter option.  
Current function value: -1.063398

```
fprintf('Optimal: (%.4f, %.4f), f = %.4f\n', x_opt(1), x_opt(2), fval);
```

Optimal: (0.5938, 1.2750), f = -1.0634

## question 2

```
clc; clear; close all;
```

```
f = @(x) (x(1)-1)^2 + (x(2)-2)^2;  
x0 = [2, 0];
```

```
[x_opt, fval] = fminsearch(f, x0, optimset('TolX', 0.01));
```

```
fprintf('Optimal: (%.4f, %.4f), f = %.4f\n', x_opt(1), x_opt(2), fval);
```

Optimal: (1.0037, 2.0006), f = 0.0000

## question 3

```
clc; clear; close all;
```

```
f = @(x) sin(x(1)) + cos(x(2)) + (x(1)-2)^2 + (x(2)-3)^2;  
x0 = [0.5, 1];
```

```
[x_opt, fval] = fminsearch(f, x0, optimset('TolX', 0.8));
```

```
fprintf('Optimal: (%.4f, %.4f), f = %.4f\n', x_opt(1), x_opt(2), fval);
```

Optimal: (2.3521, 3.0425), f = -0.1593